3. *meteorApp.css* is a CSS file to define styles

To run the app, first navigate to the app directory and run the Meteor command, as follows:

```
$ cd meteorApp
$ meteor
```

Now open the browser and enter the URL *http://localhost:3000*, where you will see output as shown in Figure 1.

Figure 1: Default Meteor output

Structure

By default, any JavaScript file found by Meteor is automatically loaded and run by it, both on the client as well as the server side. This can be a serious issue in terms of security and hence organising the code into appropriate directories is very important. Meteor provides a special way to have control of the JavaScript code to be loaded on the client and the server by using special directories which isolate the code on the server and the client side. The following is a list of special directories in Meteor:

/client

All the files in this directory are sent to the client and are published to the browser. HTML, CSS and related JavaScript can reside here.

/server

Files within this folder are run on the server only.

/public

Files in this directory are served to the client. Assets like images and videos can reside in this directory which serves as the root for assets, i.e., when including files from this directory the directory name */public* is not a part of the URL.

/private

Files within this folder can be used by the server using the assets API.

If any HTML and CSS file is encountered outside these directories, it's sent to the client side while the JavaScript gets loaded both on the client and the server side.

Templates

Templates are nothing but Meteor's way of specifying HTML. Clients render HTML through templates. Templates also serve as a connection between the interface and the JavaScript code. All templates in Meteor are defined inside the template tag.

Here's an example:

```
<template name="foo">
.
..
...
</template>
```

With the *name* attribute, we define a name for the template by which it is referred.

For every defined template, Meteor creates a template object. Everything inside a *<template>* tag is compiled into Meteor templates, which can be included inside HTML with *{{> templateName}}* or referenced in your JavaScript with *Template.templateName*.

Let us proceed to our first template by issuing the following code:

```
<template name="foo">
<h1> Welcome to Meteor</h1>
</template>
```

Now we need to include this template inside the interface. To make the 'foo' template appear on the page, place the following line inside the *.html* file:

```
{{> foo}}
```

Given below is an example of how to create and embed the template inside the HTML body.

```
<head>
        <title> Meteor </title>
</head>
<body>
        <h1> Hello World </h1>
        {{> foo}}
</body>
<template name="foo">
<h1> Welcome to Meteor</h1>
</template>
```

To view the above code in the browser, first run the Meteor server.

In the browser, enter the URL *localhost:3000* and you will see the results, as shown in Figure 2.

Templates and JavaScript

Templates can become more dynamic with the use of helpers and they can be referenced in JavaScript using *Template*.

Figure 2: Defining template